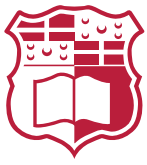
A Comparative Study of Visuomotor Policies and Latent World Models in Dexterous Manipulation

Graham Pellegrini

Supervisor: Prof. Alexiei Dingli

[PLACEHOLDER: Submission Date]

Submitted in partial fulfilment of the requirements
for the degree of Master of Science in Artificial Intelligence.



L-Università ta' Malta
Faculty of Information &
Communication Technology

Plagiarism Declaration

I, the undersigned, hereby declare that the work contained in this dissertation is my own original work and that I have not previously in its entirety or in part submitted it at any university for a degree.

I declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where due acknowledgement is made in the text.

I declare that the use of artificial intelligence tools in the preparation of this dissertation was limited to writing assistance, code suggestions, and literature search support. All research design, experimental decisions, implementation choices, result interpretation, and academic claims are my own.

Signature: _____

Name: Graham Pellegrini

Date: _____

Contents

List of Figures

List of Tables

List of Abbreviations

ACT Action Chunking Transformer.

BC Behaviour cloning.

CVAE Conditional variational autoencoder.

DDIM Denoising Diffusion Implicit Models, an accelerated inference scheme for diffusion models.

DP Diffusion Policy, a visuomotor policy that represents actions as a conditional denoising diffusion process.

I-JEPA Image Joint Embedding Predictive Architecture.

JEPA Joint Embedding Predictive Architecture.

LeWM LeWorldModel, a latent world model trained end-to-end from random initialisation.

SSL Self-supervised learning.

V-JEPA2 Video Joint Embedding Predictive Architecture 2.

V-JEPA2-AC Action-Conditioned Video Joint Embedding Predictive Architecture 2.

VAE Variational autoencoder.

WACT World Model Augmented Action Chunking Transformer.

1 Background

This chapter assumes familiarity with the basic mechanics of a neural network and introduces six concepts central to this work: visuomotor policies, rigid-body pose in $\mathrm{SE}(3)$, latent representations, conditional VAEs, transformer attention, and self-supervised learning. These foundations support understanding of the specific architectures in Chapter 2.

1.1 Visuomotor policies

A visuomotor policy is a learned function that takes a visual observation as input and produces an action or sequence of future actions as output. In a manipulation setting, the visual observation is typically a single camera frame showing the hand and the object being manipulated, and the action describes how the hand should move next. The policy is called visuomotor because it connects vision directly to motor output, without requiring a hand-crafted intermediate representation of what the scene contains [1].

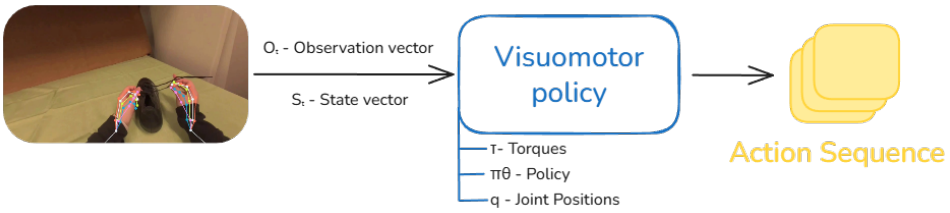


Figure 1.1 A visuomotor policy maps visual observations to motor actions. The input is a camera frame showing the hand and object, and the output is a proposed action for how the hand should move next.

Learning this mapping from data is appealing because useful hand behaviour depends on subtle visual cues that are difficult to specify by hand. The timing of a reaching movement, the grip shape adopted before contact, and the postural adjustments made in response to object geometry all depend on information visible in the camera image but difficult to encode as explicit rules [2–4]. A visuomotor policy learns these associations directly from recorded demonstrations, capturing the relationship between what the camera sees and what the hand does without requiring the programmer to describe it explicitly.

1.1.1 Behaviour cloning and compounding error

The most direct way to train a visuomotor policy from demonstrations is Behaviour Cloning (BC) [1]. Here an observed frame predicts the action taken by the expert at

that moment, and the policy is trained to minimise the difference between its predicted action and the recorded expert action across all frames in the dataset [5]. The problem arises when a small prediction error causes the hand to move to a state that is slightly different from any state in the training data. Which then results in a compounding error as the policy encounters states it has not seen during training, leading to larger and larger errors over time.

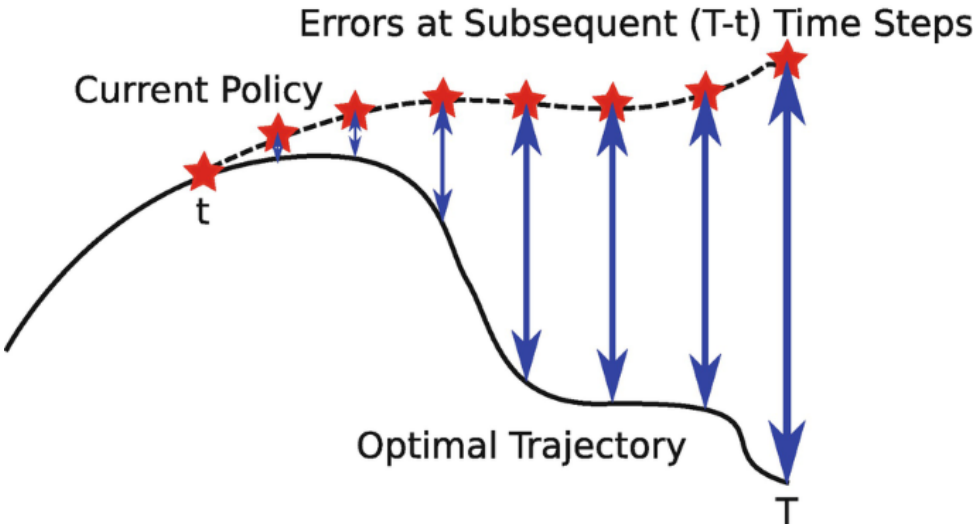


Figure 1.2 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [6].

1.1.2 Action chunking

Action chunking addresses the compounding error problem by changing what the policy predicts. Instead of predicting a single next action at each step, the policy predicts a short sequence of future actions in one forward pass. This sequence is called an action chunk [7, 8]. Hence why Figure 1.1 shows the action sequence as a stack of multiple action blocks. By committing to a chunk of X future steps, the policy makes one prediction decision every X frames rather than one per frame. With fewer decision points, there are fewer opportunities for errors to accumulate, and the hand stays closer to the training distribution for longer [9].

1.2 Rigid body pose and the $\mathrm{SE}(3)$ representation

To predict how a hand will move, a system needs a precise way to describe where the hand is and which way it is pointing. These two quantities together are called the pose of a rigid body. For the human wrist, pose is fully described by its position in space and its orientation relative to a fixed reference frame [10].

Translation – Position is represented as a vector of three numbers, one for each spatial dimension: how far the wrist is to the right or left, how far forward or backward, and how far up or down [2]. This vector $\mathbf{t} = [t_x,\ t_y,\ t_z]^\top$ is straightforward to predict with no special constraints on the output.

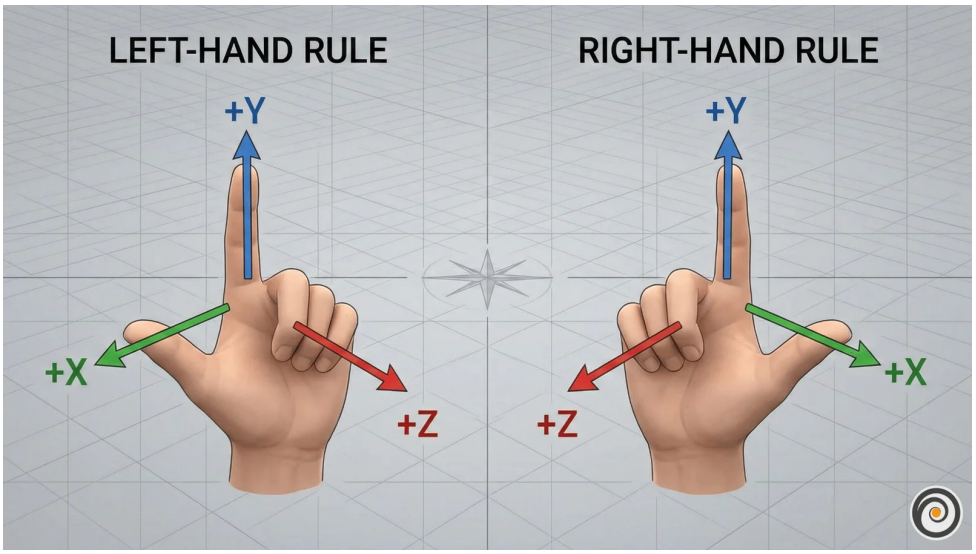


Figure 1.3 Translation for each arm can be represented by a movement along each of the three spatial axis [11].

Rotation – Orientation requires more care. A common representation uses three angles known as Euler angles, one per axis. These are intuitive but problematic for learning. At certain configurations the axes align and the system loses a degree of freedom, a singularity called gimbal lock. A subtler problem is discontinuity: a wrist rotated 359 degrees and one rotated 1 degree are almost physically identical, yet their angle values are numerically far apart. A network minimising prediction error will produce large errors near these boundaries even when the true motion is smooth [12].

A rotation matrix R avoids both issues: small physical rotations produce small numerical changes in the matrix entries, and there are no singularities.

The homogeneous transform – Translation and rotation are combined into a single 4×4 matrix, the standard representation of a pose in the special Euclidean group $\mathrm{SE}(3)$:

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{1.1}$$

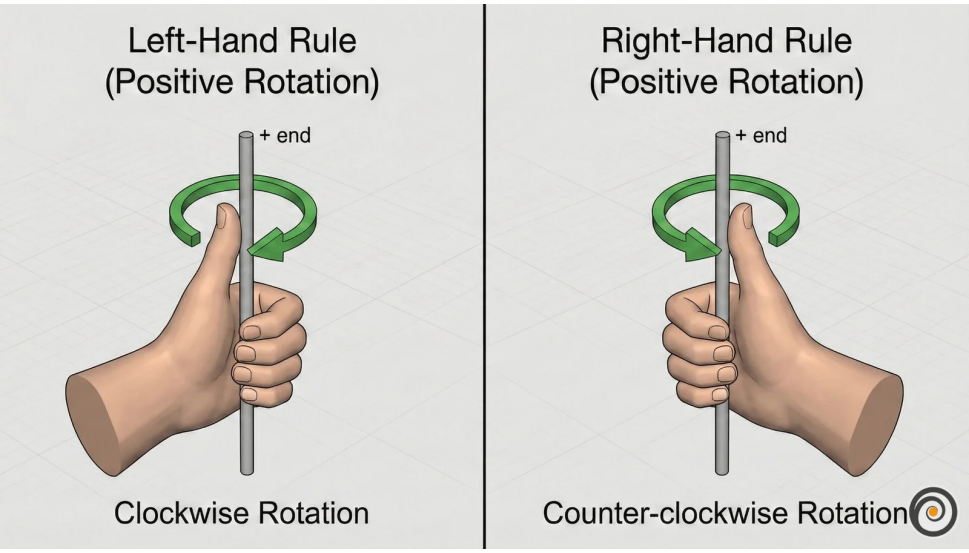
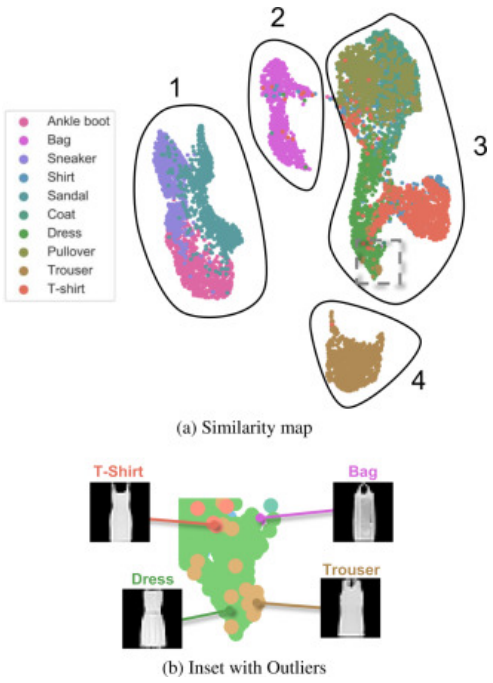


Figure 1.4 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [11].

1.3 Latent representations

A latent representation is a learned internal description of an observation. An image starts as a grid of pixel values, where each pixel stores three or four channel intensities (typically red, green, and blue, with an optional transparency value). A model does not usually reason with these raw numbers directly. Instead, it transforms them through several layers into a new set of values that no longer correspond to individual pixel coordinates or colours. These new values form a compact encoding that captures patterns in the image such as shape, position, motion, and spatial structure. The collection of these encodings is often called the latent space. Figure 1.5 shows an example in which similar items cluster together in the learned representation.



(a) Similarity map

(b) Inset with Outliers

Figure 1.5 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [13].

The purpose of this transformation is to keep task-relevant structure while discarding unnecessary visual detail. For a manipulation scene, a useful latent state might encode hand position, object pose, contact progression, or motion trend without preserving every texture or lighting variation. A latent state is therefore not directly interpretable in the way an image is, but it can still preserve the information needed to reason about what happens next.

This is why latent prediction is often a better fit than pixel-level prediction for high-dimensional video observations. Predicting the next frame at the pixel level requires a model to account for every visual detail of the scene, including lighting changes, texture variation, and background movement that carry no information about task-relevant dynamics. A latent predictor instead operates on compact embeddings and concentrates predictive capacity on the structure that determines what actually happens next [14]. This design trade-off motivates the world model architectures reviewed in Chapter 2.

1.4 Conditional variational autoencoders

A VAE maps an input to a distribution of latent vectors described by a mean μ and standard deviation σ, rather than a single point. Sampling different latent vectors lets the decoder produce different but plausible outputs rather than one fixed answer. Figure 1.6a shows this standard encoder-latent-decoder flow [15].

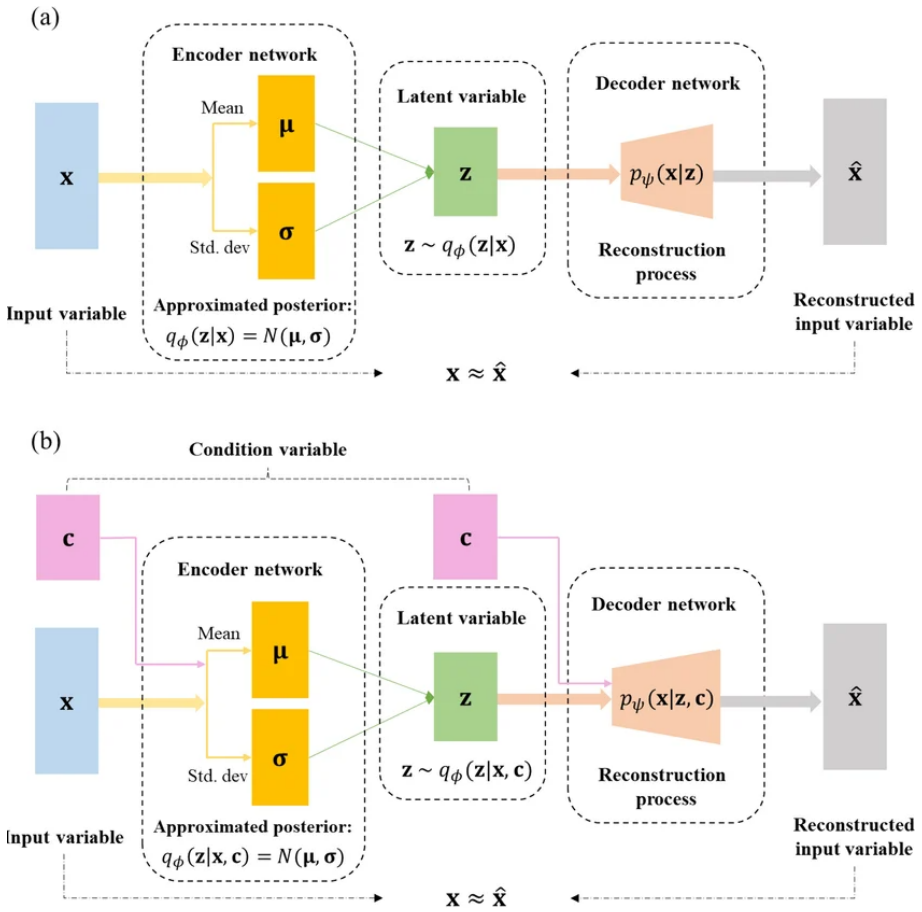


Figure 1.6 Standard VAE (a) and conditional CVAE (b) encoder-decoder pipelines, adapted from [16].

For the policy to be trainable end-to-end, gradients must flow from the action prediction loss all the way back to the encoder. Sampling z directly from a distribution blocks this path. The reparameterisation trick resolves this by writing $z = \mu + \sigma\epsilon$, where ϵ is standard normal noise drawn independently of the learned parameters. Figure 1.7 shows the full encoder-latent-decoder pipeline alongside its training loss: a reconstruction term that rewards accurate output and a regularisation term that keeps the latent distribution close to a standard Gaussian, together forming the evidence lower bound [15].

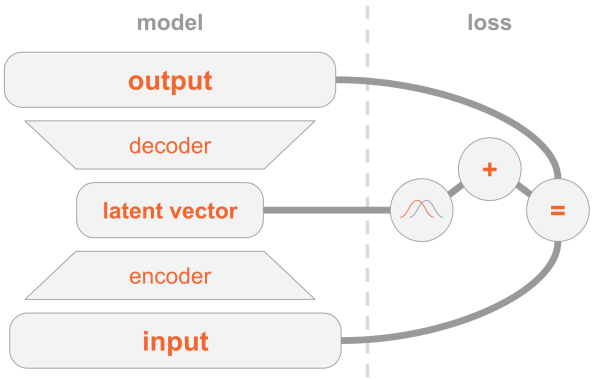


Figure 1.7 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation), adapted from [17].

A CVAE adds an extra conditioning input to both encoder and decoder, shown in Figure 1.6b. In a visuomotor policy, this condition is the current observation frame, so the latent variable only needs to capture how the future action varies within that scene [18]. The Action Chunking Transformer (ACT) architecture uses this to sample multiple latent vectors from one observation and decode each into a candidate action chunk [7].

1.5 Transformer networks and the attention mechanism

A transformer processes a sequence as a set of tokens. A token can represent a word, an image patch, a timestep in an action sequence, or any vector produced by an earlier model component. The architectural idea needed here is simple: each token is updated by looking across the other tokens and deciding which of them are most relevant. This relevance-weighted exchange of information is called attention [19].

Attention forms three learned views of every token:

- **Query** (Q) – what this token is looking for.
- **Key** (K) – what another token contains.

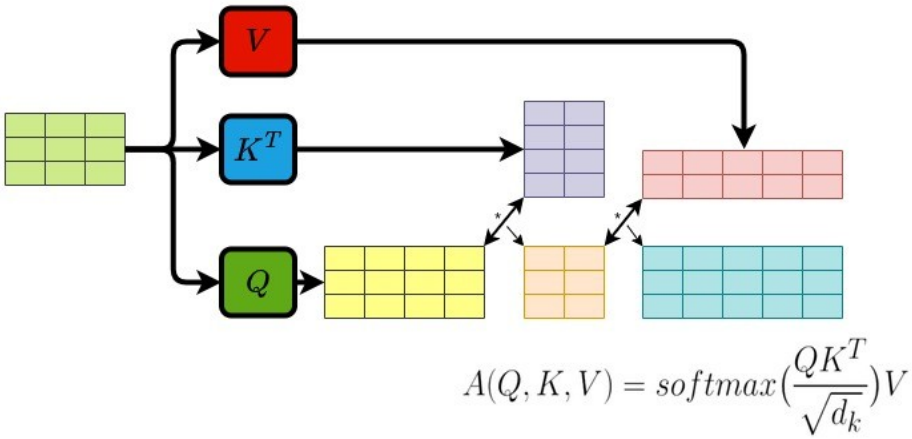


Figure 1.8 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [20].

- **Value** (V) – what another token contributes if attended to.

Figure 1.8 shows how each token (highlighted) sends its query outward to every other token, receives weighted values in return, and produces an updated output. The bidirectional arrows indicate that every token attends to every other token simultaneously, and the crossing arrow beneath represents the weighted sum that combines those values into the final output. For each token, the transformer compares its query with the keys of all tokens. The resulting scores become weights, and those weights are used to combine the corresponding values. This operation is usually written as [19]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{1.2}$$

Transformers usually repeat this operation in several heads and add positional information so the model knows where each token sits in the sequence. Multiple heads allow different relationships to be represented at the same time, while positional information preserves order [19].

1.6 Self-supervised learning

Machine learning methods can be grouped by how they receive feedback during training, as illustrated in Figure 1.9. Supervised learning trains on data with explicit labels: the correct answer is provided for every input, and the model learns to reproduce it. BC from Section 1.1.1 is a direct example: the expert demonstration acts as the label. Reinforcement learning instead receives feedback through a reward signal

after taking actions in an environment; the model learns which state-action sequences lead to good outcomes. SSL sits between these two: it trains on unlabelled data by having the data itself provide the supervision signal. Part of the input is hidden, and the model learns to predict it from the remainder [21].

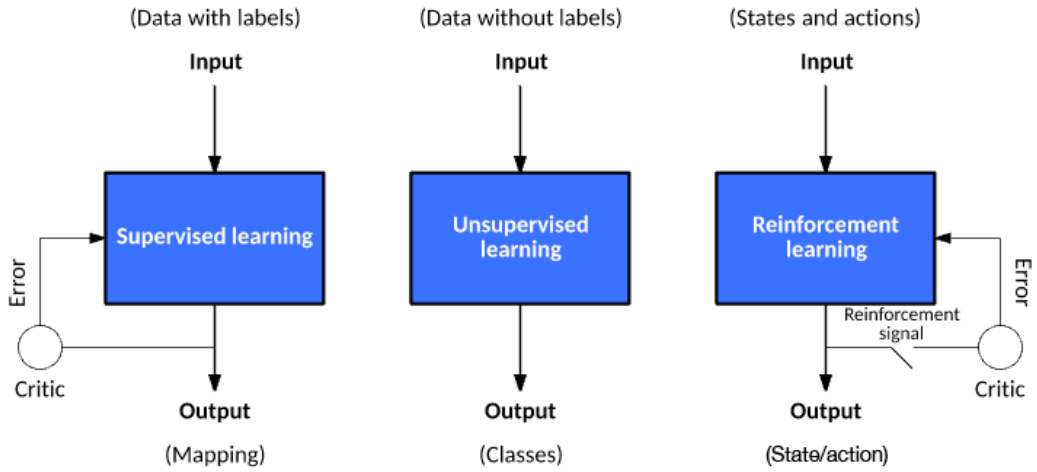


Figure 1.9 Three machine learning paradigms: supervised, reinforcement, and self-supervised learning [22].

Applied to video, SSL takes the following form: given a sequence of observed frames, hide one or more future frames and ask the model to predict them from the frames that came before. No human annotation is required because the video recording itself provides the hidden target.

Predicting raw pixels runs into a problem described in Section 1.3: when the future is ambiguous the model cannot commit to a single outcome, so it predicts the average of all possibilities and produces a blurred image that does not match any real frame. Predicting in latent space avoids this. Instead of reconstructing every pixel, the model predicts the compact embedding of the future frame. Background motion and fine texture are discarded at the encoding step, so the predictor focuses on the structure that determines what actually happens next [14]. When the predictor is also given the action the agent intends to take, the result is a world model: a learned function that predicts how the scene embedding will change in response to a proposed action. The architectures that build on this idea are reviewed in Chapter 2.

2 Literature Review

This chapter surveys the literature motivating the three central design choices of World Model Augmented Action Chunking Transformer (WACT): that a specialist imitation policy is a strong base for dexterous manipulation, that a world model can usefully evaluate candidate action chunks in an offline benchmark, and that latent predictive learning is the right approach for building such an evaluator. Sections 2.1 and 2.2 review the two candidate imitation policies, ACT and Diffusion Policy, examining both the challenges they address and the trade-offs between them. Section 2.3 introduces world models as learned environment predictors, surveys the evaluator architectures most relevant to this role, and covers the three latent predictive architectures implemented in the benchmark, together with a negative control. Section 2.4 covers offline evaluation frameworks and their validity as proxies for real-world performance. The chapter closes with a summary of the literature gap that the benchmark addresses. The chapter assumes familiarity with the technical foundations in Chapter 1; world models are introduced here.

2.1 Action Chunking with Transformers (ACT)

ACT was introduced by Zhao et al. to address the compounding error and multimodal distribution challenges that make dexterous bimanual manipulation demanding for imitation learning [7]. On the ALOHA bimanual platform it achieved success rates of up to 90% on precision tasks such as threading a cable tie, inserting a battery, and opening a translucent bag, outperforming behaviour cloning baselines and GATO by substantial margins [7]. In SE(3) wrist-pose prediction specifically, small deviations in predicted position or orientation alter contact geometry in ways that cascade into grasp failure [1], and a policy trained with a naïve regression loss produces averaged predictions that may not represent any plausible trajectory when the demonstrated distribution is multimodal [8]. ACT's CVAE architecture addresses both of these failure modes directly.

ACT addresses both challenges through the use of a CVAE design (Section 1.4). The respective CVAE has an encoder-decoder structure depicted in Figure 2.1, where the encoder compresses an action sequence and joint observation into a style variable z that captures the multimodal distribution of plausible futures, and the decoder synthesizes images from multiple viewpoints, joint positions, and z with a transformer encoder, and predicts a sequence of actions with a transformer decoder. The encoder is discarded at test time, and z is simply set to the mean of the prior (i.e. zero) at test time [7].

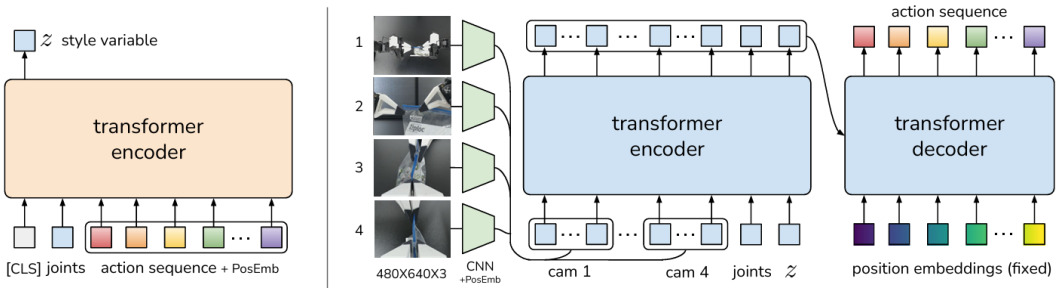


Figure 2.1 Architecture of the Action Chunking with Transformers (ACT) policy [7].

At inference time, the CVAE decoder samples several latent vectors independently and decodes each into a candidate action chunk. A temporal ensembling technique is then applied: consecutive chunks overlap in their predicted time horizons, and a recency-weighted average is taken across overlapping predictions to produce the final action. This averaging smooths the executed motion and reduces jerkiness caused by abrupt chunk transitions. The result is a structured pool of plausible futures, all conditioned on the same observation, from which the ensembled output is drawn [7].

ACT carries known limitations alongside these strengths. The Gaussian prior on the style variable z constrains how faithfully the CVAE can represent highly multimodal action distributions: at transitions where qualitatively different trajectories are equally plausible, the latent space can average across modes rather than sample cleanly from each one, producing trajectories that are plausible in aggregate but imprecise at the most ambiguous decision points [8]. Chunk length is a fixed hyperparameter that must be tuned per task; longer chunks improve smoothness but commit the policy to a trajectory over a horizon that may no longer be appropriate as the scene evolves. Temporal ensembling introduces a recency-weighted lag that can slow the policy's response to sudden environmental changes [7]. These properties are well-documented in follow-on work comparing ACT with more expressive policy classes, and they motivate the inclusion of Diffusion Policy as the second candidate in the benchmark.

2.2 Diffusion Policy

Diffusion Policy offers a principled alternative approach to the same compounding error and multimodality challenges. Its core concept relies on starting from a noise distribution and iteratively refining action predictions through a diffusion process. By gradually denoising the predictions, Diffusion Policy captures complex multimodal distributions effectively, producing plausible trajectories that align with the demonstrated data [8]. Figure 2.2 illustrates the policy overview:

- a) General formulation: at timestep t, the policy takes the latest T steps of

observation data O_t as input and outputs T steps of actions A_t.

- b) CNN-based Diffusion Policy: FiLM (Feature-wise Linear Modulation) [23] conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from K_t drawn from Gaussian noise, the output of noise-prediction network ϵ_θ is subtracted, repeating K times to get A_t^0, the denoised action sequence.
- c) Transformer-based Diffusion Policy: the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated [24, 25].

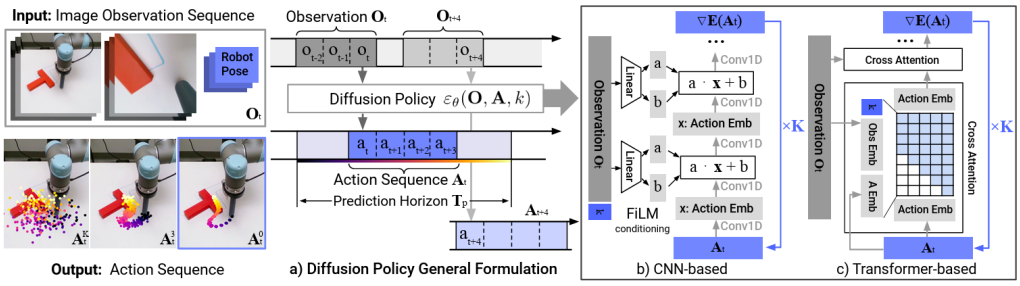


Figure 2.2 Diffusion Policy: general formulation (a), CNN variant (b), transformer variant (c) [8].

Chi et al. report that Diffusion Policy outperforms ACT on tasks with high action multimodality (such as multi-step object rearrangement and cup placement under visual ambiguity) while matching it on simpler reaching tasks [8]. The advantage arises because iterative denoising explores the full action distribution at each step rather than committing to a single latent mode as the CVAE prior must, making it more robust to the mode-averaging failure that affects ACT at ambiguous transitions.

The main limitation of Diffusion Policy is inference cost. Generating a single action chunk requires multiple sequential network evaluations: standard DDPM variants use on the order of 100 denoising steps compared with the single forward pass of ACT's decoder, which substantially reduces the achievable control frequency [8]. Efficient transformer variants address this through progressive noise scheduling and parameter sharing, recovering much of the inference speed at added architectural complexity [25]. Both policies are included in the benchmark to span this representational trade-off: ACT's structured latent sampling is computationally efficient but expressively constrained at high multimodality, while Diffusion Policy's iterative denoising is more expressive but more demanding at inference. Measuring the world model's effect across both allows the benchmark to determine whether evaluator gains are consistent across this axis.

2.3 World models

A world model is a learned predictor of how a system evolves over time. Given the current state and a proposed action, it estimates the state that is likely to follow. In a manipulation setting, the state may be a raw image, a latent representation of the scene (Section 1.3), or a structured description of hand and object configuration. Regardless of the representation chosen, the underlying principle is the same: the model internalises the dynamics of the environment and uses them to project forward from any given state [26].

World models become especially useful when a pool of candidate actions must be evaluated before any one is committed to. Each candidate is rolled forward through the model, producing a predicted future state; those predictions are ranked by predicted cost or consistency. The quality of this ranking depends heavily on the chosen representation: pixel-space world models must reconstruct every visual detail and suffer from mode-averaging at multimodal transitions, while models that operate in the compact learned latent space introduced in Section 1.3 can concentrate predictive capacity on the dynamics that matter for task outcomes [14]. The following subsections survey the literature supporting this evaluator design and describe the three world model architectures implemented in the benchmark.

2.3.1 World models as policy evaluators

WorldGym demonstrates that a learned world model used as a simulated environment for policy evaluation produces rankings that align with real-world task success with meaningful fidelity [27]. Candidate policies are evaluated not by deploying them physically but by rolling them out inside the model's predicted environment, reducing the cost of policy comparison to a forward pass. CLOVER extends this to closed-loop control, as shown in Figure 2.3. A visual planner generates a sequence of sub-goal frames; the executor measures the embedding-space error between the predicted sub-goal and the current observation. When the error exceeds a threshold, the system triggers an adaptive transition and replans rather than committing to the original trajectory. The feedback loop, absent from open-loop imitation policies, is the mechanism that translates world model prediction accuracy into task completion gain [28]. Together, WorldGym and CLOVER establish that world model rankings are informative both before and during execution.

The broader question of whether such evaluator rankings transfer to physical outcomes, and the qualifications around out-of-distribution reliability, are examined in Section 2.4.

The practical value of this evaluator loop is measurable. Table 2.1 reproduces

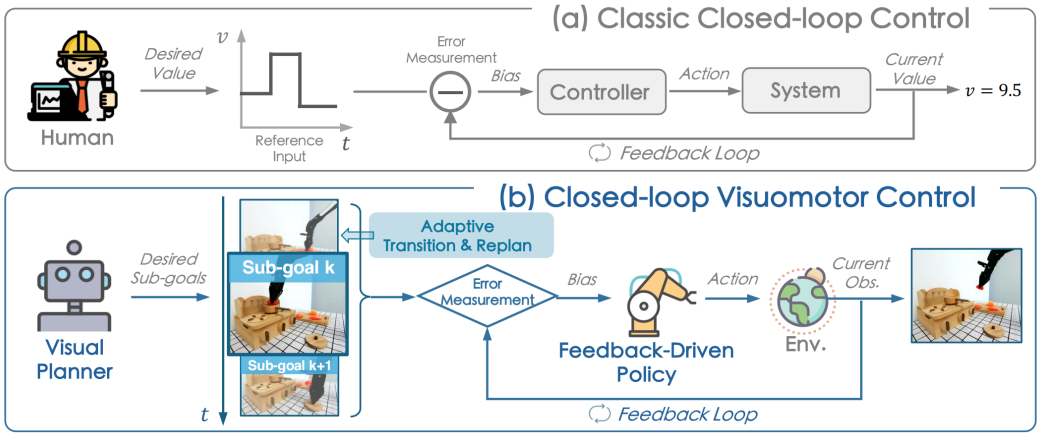


Figure 2.3 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [28].

CLOVER's real-world evaluation on a multi-step robot manipulation suite, reporting the average number of consecutive sub-tasks completed (`avg_len`) for representative baselines and CLOVER [28].

Table 2.1 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [28].

Method	avg_len
ACT [7]	0.6
R3M (pretrained visual repr.) [29]	0.7
RT-1 [30]	1.1
CLOVER (closed-loop)	**2.1**

Alongside the task completion gain, CLOVER reduces embedding-space prediction RMSE relative to open-loop baselines by correcting the policy's course when the world model detects a divergence from the expected trajectory, demonstrating that prediction accuracy and task success move together under the evaluator framing.

2.3.2 Latent predictive architectures

The energy-based framework of LeCun and Dawid establishes why prediction should operate in latent rather than pixel space [14]: the same representational argument introduced in Section 1.3 applies directly to the prediction target, favouring compact embeddings over pixel reconstructions that force the model to account for irrelevant visual detail and average over all plausible continuations at ambiguous transitions.

Image Joint Embedding Predictive Architecture (I-JEPA) establishes the joint-embedding predictive architecture as a viable approach to learning rich visual

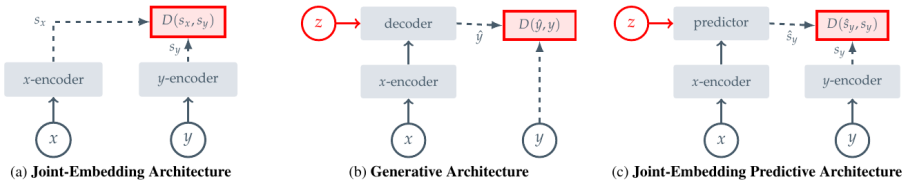


Figure 2.4 Three SSL prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [21].

representations without any pixel reconstruction objective [21]. Panel (c) of Figure 2.4 shows the data flow directly: a context block x from the image is passed through the context encoder to produce latent tokens z_x. A lightweight ViT predictor then takes z_x together with positional tokens indicating where the masked target regions y are located, and outputs predicted latent representations $\hat{s}_y$ for those regions. A separate target encoder, whose weights are a slow exponential moving average of the context encoder, independently encodes the actual target pixels into ground-truth latents s_y. The loss $\text{MSE}(\hat{s}_y, s_y)$ is computed entirely in latent space: no decoder, no pixel generation, no reconstruction of lighting or texture. The outcome is a context encoder whose representations capture semantically meaningful structure (object shape, spatial layout, and scene dynamics) because that is all the predictor ever needs to get the target embeddings right. Training is approximately 2.5 times faster than pixel-reconstruction methods as a result. V-JEPA2 extends this same joint-embedding predictive architecture from images to video sequences.

Four architectures are covered in the following subsections, each representing a distinct representational strategy. V-JEPA2 and V-JEPA2-AC share the same pretrained video backbone: V-JEPA2 is the internet-scale pretrained encoder, while V-JEPA2-AC extends it with an action-conditioned transition head and is the form used directly in the benchmark. DexWM uses a pretrained image backbone with specialised hand-geometry supervision. LeWorldModel (LeWM) learns all representations from scratch on the task data itself.

V-JEPA2

V-JEPA2 extends the I-JEPA framework from static images to video, pretraining on over one million hours of internet video [31]. The pretraining shifts from spatial to spatio-temporal masking: the context encoder receives past observation frames, and the predictor must recover future latent states. The training objective combines teacher-forcing with multi-step autoregressive rollout losses, encouraging accurate short-horizon predictions and consistent long-horizon trajectory representations. The result is a frozen ViT-Large encoder, approximately 300 million parameters, that encodes physical priors about hand and object interactions acquired from

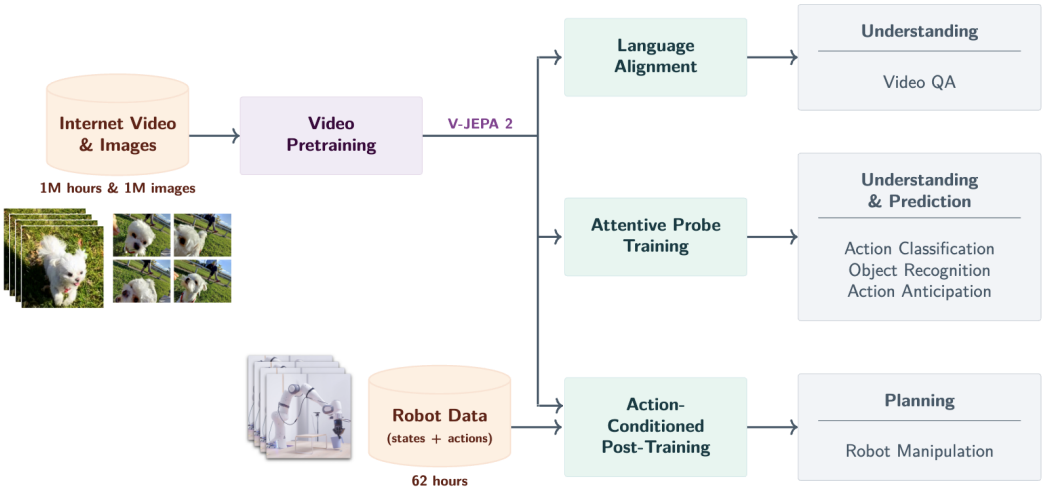


Figure 2.5 V-JEPA2 post-training pathways from internet-scale backbone to specialised applications [31].

internet-scale pretraining. As shown in Figure 2.5, the pretrained backbone supports three post-training pathways: Language Alignment, which grounds visual embeddings in natural language; Attentive Probe, which adds a lightweight classification head; and Action Conditioning, which extends the predictor to take robot actions as input and forms the basis of V-JEPA2-AC.

V-JEPA2-AC

The frozen ViT-Large encoder from V-JEPA2 serves as the backbone of V-JEPA2-AC, producing a fixed latent embedding for each observation frame with no gradient updates to the backbone weights. A lightweight action-conditioned transition head is trained on top of these stored embeddings: it takes the current frame latent together with a proposed action chunk (robot joint states and end-effector poses) as the conditioning signal z, and predicts the latent representation of the future frame. Candidate action chunks are then scored by comparing the predicted future embedding against the actual observed future embedding under an L1 objective. This design, shown on the right of Figure 2.6, separates what the network must learn into two parts: the frozen encoder supplies rich visual features that generalise from internet-scale pretraining, and the small trained head learns only how actions propagate through those features.

An important qualification is raised by Tsagkas et al., who show that pretrained visual representations can encode spatial plausibility well while providing a weaker signal for fine-grained temporal dynamics [32]. A frozen encoder trained on internet video may represent hand shape and position accurately at each individual frame while providing a less reliable signal about whether the predicted trajectory is kinematically

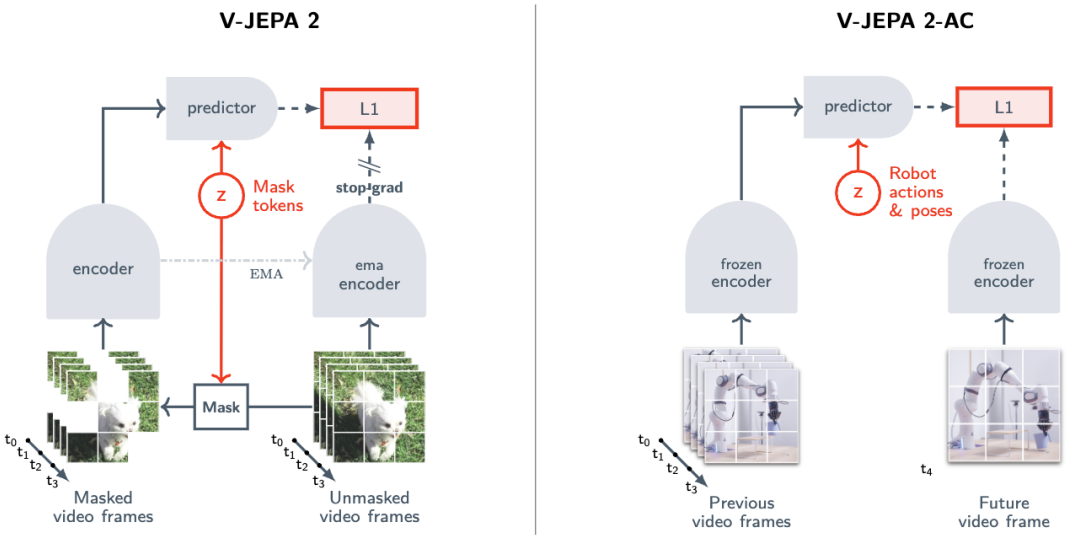


Figure 2.6 Standard V-JEPA2 (left) versus action-conditioned V-JEPA2-AC (right) [31].

consistent across the full chunk horizon. This temporal limitation provides context for the path shape results in Chapter **??**.

DexWM

DexWM [33] is a dexterous manipulation world model from Meta FAIR. As shown in Figure 2.7, its pipeline encodes each input frame with a frozen DINOv2 image encoder, producing a grid of spatial patch tokens rather than a single global embedding. These patch-level representations preserve fine-grained spatial information about hand positions and object locations, which gives the predictor strong generalisation across diverse environments and viewpoints. A Conditional Diffusion Transformer (CDiT) predictor then takes the spatial latent tokens together with action and camera-motion inputs to predict the next latent state, and a lightweight decoder maps the result back to image and keypoint space. Where V-JEPA2-AC uses a video-pretrained backbone, DexWM uses a backbone pretrained on static images; the comparison therefore isolates the contribution of video-specific temporal pretraining to world model quality.

A distinguishing feature of DexWM is the Hand Consistency (HC) loss: an auxiliary transformer head predicts fingertip and wrist heatmaps from the latent state, imposing a geometric prior that the predictor's latent trajectory must remain consistent with plausible hand configurations. This explicit hand-geometry supervision is absent from both V-JEPA2-AC and LeWM, whose latent spaces are shaped only by prediction error or general regularisation. Goswami et al. report a 50% improvement over Diffusion Policy on grasping and placing tasks, with an 83% zero-shot real-world grasping success rate under latent planning [33].

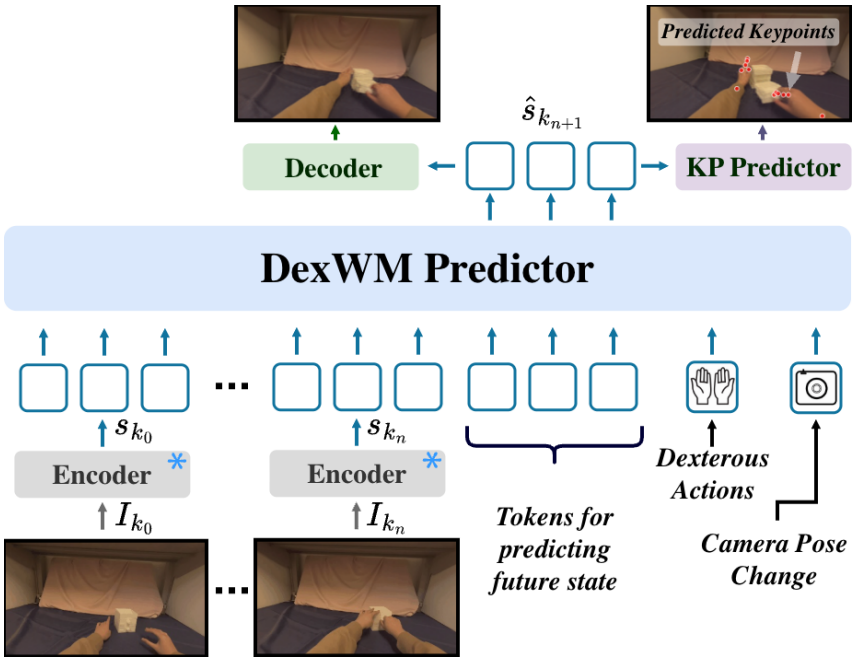


Figure 2.7 DexWM world model pipeline [33].

LeWorldModel

LeWM is a latent world model trained end-to-end from random initialisation on raw pixel observations [34]. Its architecture consists of two components: an encoder that maps each frame o_t to a compact latent vector z_t, and a predictor that models environment dynamics by estimating the next latent embedding $\hat{z}_{t+1}$ from z_t and the action a_t. The encoder is a ViT-Tiny with approximately 15 million parameters. Each frame is represented as a single 192-dimensional token rather than a spatial patch grid, which keeps the latent space compact and makes planning significantly faster than models that produce higher-dimensional patch representations.

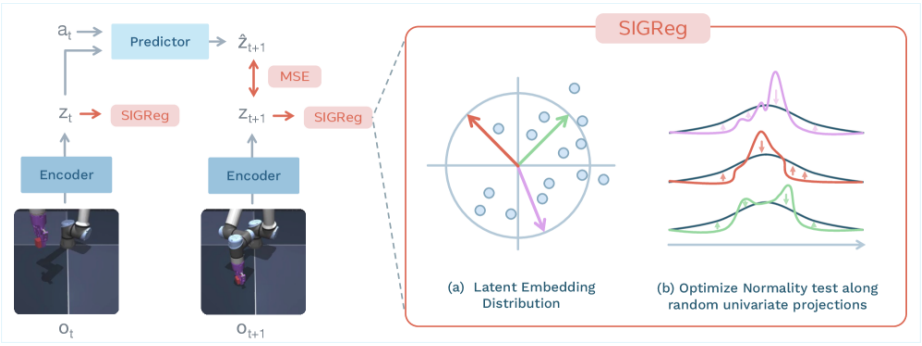


Figure 2.8 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [34].

As shown in Figure 2.8, the training pipeline consists of an encoder that maps each observation frame to a compact latent vector, a dynamics predictor that autoregressively estimates the next latent given the current embedding and action, and

a SIGReg regulariser that enforces a Gaussian prior on the latent distribution by optimising normality along random univariate projections. The training objective reduces to two terms:

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda \, \text{SIGReg}(Z) \tag{2.1}$$

where $\mathcal{L}_{\text{pred}}$ penalises inaccurate next-embedding forecasts and SIGReg(Z) stabilises end-to-end training without contrastive negative pairs or an exponential moving average target encoder [34]. This reduces the number of tunable loss hyperparameters from six, as required by prior Joint Embedding Predictive Architecture (JEPA)-style designs, to one.

DINO-WM, the closest comparable baseline, follows a similar latent prediction objective but uses a frozen DINOv2 backbone producing a grid of spatial patch tokens per frame [35]. Like DexWM, DINO-WM therefore carries the spatial resolution and generalisation benefits of DINOv2's patch-level representations, but at substantially higher computational cost. LeWM replaces the frozen backbone with a fully learned ViT-Tiny encoder that collapses each frame to a single 192-dimensional token, trading spatial resolution for training speed. In practice, LeWM can be trained on a single GPU in a few hours and achieves a 48$\times$ planning speedup over DINO-WM [34]. DexWM's CDiT predictor operating on full patch grids is significantly slower to train and requires substantially more GPU memory; this difference in training cost is expected to widen further at larger dataset scales and is quantified in the implementation chapter. Despite its compact size, LeWM's latent space encodes physically meaningful structure: the model detects physically implausible state transitions and captures spatial information such as agent and object locations across frames.

The three-way comparison across V-JEPA2-AC, DexWM, and LeWM is therefore not simply an architecture comparison. It is a comparison between fundamentally different representational strategies: video-pretrained transfer (V-JEPA2-AC), image-pretrained transfer with geometric supervision (DexWM), and self-supervised task-data learning from scratch (LeWM). This distinction makes the comparison principled and is the basis for the experimental design in Chapter 3.

PointWorld

PointWorld is a large pretrained 3D world model for in-the-wild rigid arm manipulation [36]. Unlike the latent-space models described above, it operates directly in 3D geometry: scene state is lifted from RGB-D images into a full-scene point cloud, and robot actions are represented as the 3D point flows traced by robot surface geometry through forward kinematics. This embodiment-agnostic representation allows the model to reason over physical interaction geometry rather than learned

abstract features.

Such a model is not designed to handle a 16-dimensional dexterous hand action space, where fine-grained finger articulation drives the manipulation outcome. The forward-kinematics representation assumes rigid arm geometry, making the model structurally mismatched when applied to dexterous bimanual hand motion.

2.4 Offline evaluation

A central concern for any offline benchmark is whether its metrics produce rankings that are meaningful proxies for real-world performance. Without this property, an offline evaluation framework reduces to a measure of in-distribution accuracy with no predictive validity for deployment.

At large scale, DreamerV3 demonstrates the effectiveness of latent-space candidate evaluation across diverse continuous control domains, including locomotion and manipulation [26]. DreamerV3 is an online reinforcement learning system, not an offline benchmark; it is included here because it provides large-scale empirical evidence that latent-space evaluation is an effective selection mechanism when properly calibrated, which motivates its use in the offline setting of WACT. Its world model maintains a compact recurrent latent state that transitions under action conditioning; candidate trajectories are rolled out entirely inside this imagined space and scored by a learned value function, with no return to the real environment between evaluations. TD-MPC2 confirms the principle with a scalable planner that optimises a temporal-difference value function defined over predicted latent trajectories rather than observed states [37]. Both systems show that ranking candidate actions by predicted latent-space outcomes is competitive with direct online evaluation across a range of motor control benchmarks.

A critical qualification is that latent-space scoring becomes unreliable outside the training distribution. MOPO, MOReL, MBOP, and MOPP each establish that learned models produce inaccurate predictions for out-of-distribution states, requiring conservative candidate selection to avoid trajectory degradation [38–41]. None of these works addresses dexterous manipulation specifically; they benchmark locomotion and simple control tasks, leaving open how well the principle transfers to the fine-grained wrist-pose prediction setting.

Two further bodies of work establish that offline rankings can transfer to physical outcomes when the evaluation domain is well-calibrated.

SimplerEnv establishes the correspondence between simulation-based evaluation and physical robot performance for visuomotor policies [42]. The study evaluates multiple manipulation tasks and policy types in simulation and then

physically, using a WidowX robot under matched visual and control conditions. Rankings produced from simulation-only evaluation closely track rankings from physical trials, with correlations holding across both policy families and task types. The key condition for transfer is domain calibration: visual appearance, camera viewpoint, and control interface must match between the simulation and the physical system. When these are well-matched, simulation rankings provide a reliable ordering of policies without requiring physical deployment.

WorldEval extends the validation to the world-model setting [43]. Rather than evaluating policies in a manually constructed simulator, WorldEval scores them using a learned world model that predicts future states from offline trajectory data. The world-model-based rankings correlate strongly with real-world task success: policies that score well in world-model evaluation also tend to succeed when deployed physically.

Both results carry a shared qualification: rankings hold under well-calibrated conditions, where the evaluation domain is consistent with the training distribution and the deployment context. When this alignment breaks down, offline rankings can degrade. The design implications of this constraint are discussed in Chapter 3.

Summary. The literature surveyed in this chapter establishes three supporting findings. First, world model rankings are informative evaluators both before and during execution, as demonstrated by WorldGym and CLOVER [27, 28]. Second, latent predictive objectives produce more efficient and better-calibrated representations than pixel-level reconstruction, as demonstrated by the JEPA family [21, 31]. Third, offline evaluation rankings can carry predictive validity for physical outcomes when the evaluation domain is well-calibrated, as demonstrated by SimplerEnv and WorldEval [42, 43].

The gap these findings leave open is twofold. First, there is no controlled offline benchmark that isolates the contribution of a world model evaluator to the trajectory quality of a fixed dexterous manipulation policy. The offline reinforcement learning literature establishes the need for principled candidate selection but benchmarks only locomotion and simple control tasks [38–41]. Second, the interaction between world model representational strategy and policy performance has not been examined: it is not known whether a pretrained backbone world model, a task-data-trained world model, and an out-of-domain negative control produce meaningfully different guidance when applied to the same policy on the same data. Both gaps require a benchmark that holds the policy and evaluation surface fixed while varying only the world model. The methodology for constructing and running such a benchmark is described in Chapter 3.

3 Methodology

3.1 Dataset and Experimental Setup

3.1.1 EgoDex: source and selection rationale

The benchmark is built on EgoDex [44], a large-scale egocentric dataset of bimanual dexterous manipulation collected using Meta Quest Pro head-mounted cameras. It spans 63 task categories including folding, slotting, pouring, and kneading, covering a wide range of fine-grained hand motions across more than 194,000 demonstration episodes and 1.3 TB of paired video and annotation data. The full release comprises five training parts and supplementary additional-data splits; this study uses the first three training parts. Each episode provides synchronised RGB video and full six-degree-of-freedom SE(3) wrist annotations for both hands at approximately 30 frames per second.

The choice of dexterous manipulation as the evaluation domain is not incidental. This dissertation forms part of the University of Malta's ongoing M.AI.ESTRO research project, which investigates the use of AI for scene and task prediction on dexterous manipulation tasks to support industrial transfer learning and human skill digitisation. The research domain was therefore set by this institutional context: the methodology operates on human hand demonstrations rather than robotic end-effector data because the target application is the transfer of tacit human dexterous skill. An evaluation framework built around robot arm trajectories would not address that objective. The introduction situates this motivation more broadly; the present chapter focuses on the methodological consequences of the domain choice.

Within the dexterous manipulation setting, EgoDex was selected on three properties. First, the benchmark requires a dexterous bimanual hand action space: fine-grained articulation across a sixteen-dimensional SE(3) wrist pose representation drives the manipulation outcome, and large-scale corpora built around rigid gripper or arm contact do not provide this. Second, the egocentric viewpoint aligns with the intended downstream deployment setting of a wearable assistive device. Third, the dataset's scale and task diversity are sufficient to support simultaneous training of five world model families and two policy architectures on declared non-overlapping splits while retaining a held-out test set of meaningful size.

EgoDex carries two acknowledged limitations. The demonstrations are produced by human subjects rather than a robotic system, which introduces a kinematic domain gap between the training distribution and any robotic deployment. No contact or force signals are present; the action representation is purely kinematic

$\mathrm{SE}(3)$. Both are accepted as known constraints rather than experimental variables and are discussed further in Section 3.7.

3.1.2 Windowing protocol

The dataset is stored as paired `.mp4` and `.hdf5` episode files. A deterministic index stage discovers all valid paired episodes, infers their lengths from HDF5 metadata, and writes canonical observation and prediction windows for each split. This shared index is the single source of truth for all downstream modules: policy training, world model extraction, and evaluation all consume the same window file for their respective split, ensuring that output differences across the experimental branches can be attributed to branch design rather than to differences in the data presented to each component. Two windowing strategies are used at different stages of the project.

Strategy 1: Fixed-stride sampling. Each window spans 16 observation frames followed by 16 prediction frames, with consecutive window starts separated by a stride of 128 frames. At 30 frames per second this places starts approximately 4.3 seconds apart, corresponding to an observation context and prediction horizon of approximately 0.53 seconds each. This contract is encoded as `obs16_pred16_s128` and forms the foundation for all policy training and world model training (see Figure 3.1).

Figure 3.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic.

Strategy 2: State-change-targeted sampling. The fixed-stride scheme samples the episode timeline uniformly, which causes it to under-sample high-motion events: long near-static segments contribute windows at the same rate as active manipulation periods. As detailed in Chapter **??**, this limitation motivated a revised strategy adopted for the placement perception study and qualitative development inspection. Rather than placing windows at fixed intervals, this approach computes smoothed hand-state change scores from the wrist transform and confidence sequences, selects candidate windows near high-change peaks, and applies episode-first task-stratified sampling with a fixed random seed (see Figure 3.2). A relative-position fallback retains slower episodes that would otherwise be excluded, yielding one frozen window per episode stratified across all represented task categories.

The canonical training and evaluation index uses the fixed-stride scheme across all experimental branches. The state-change-targeted subset was used exclusively for

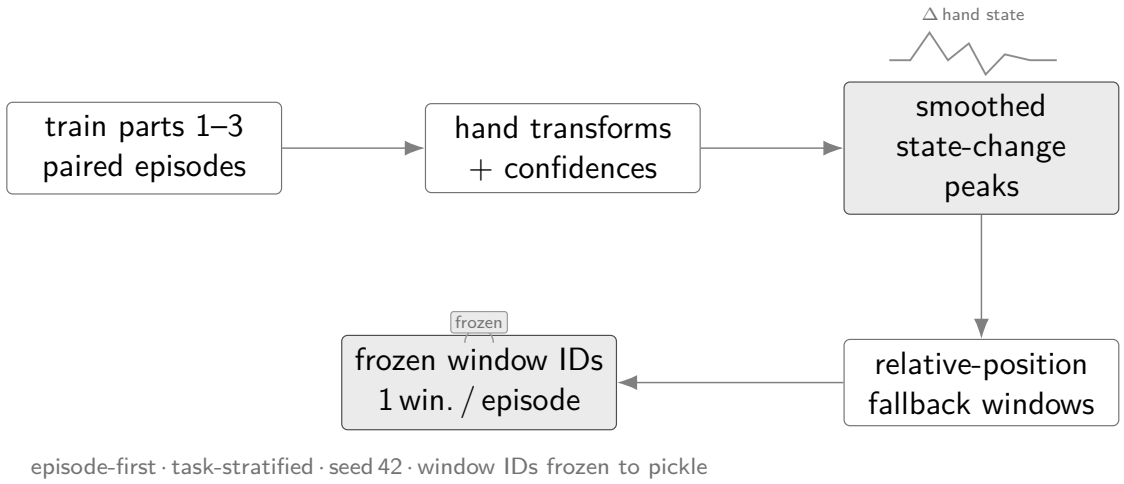


Figure 3.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic.

the placement perception study and qualitative inspection during development; it does not form part of the primary benchmark evaluation.

3.1.3 Training splits and test set

Three cumulative training splits were assembled from the first three parts of the EgoDex release, each a strict superset of the previous; Table 3.1 summarises their composition. EgoDex ships five training parts plus supplementary additional-data splits; parts four and five and all additional-data splits are excluded from this study to bound the experimental scope. The held-out test split is the official EgoDex test set and is never used during training at any stage by any model or policy.

Table 3.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index.

Split	Tasks	Episodes	Windows	Role
`train_part1`	26	46,091	140,239	Stage 1 training
`train_part1_2`	39	140,654	291,369	Stage 1+2 training
`train_part1_2_3`	63	194,333	430,962	**Canonical training**
`test`	111	3,229	7,607	**Frozen evaluation**

The episode counts reflect the uneven collection density across parts: the 13 task categories added in part two average approximately 7,300 episodes each, roughly four times the average of the 26 part-one tasks, because part two focuses on high-frequency foundational primitives such as pick-and-place and folding that were collected at greater volume.

The canonical results in Chapter **??** use `train_part1_2_3` across all trained

components. The test split is the official EgoDex held-out set spanning 111 task categories, which includes all 63 categories present in `train_part1_2_3` as well as 48 categories drawn from the excluded parts; evaluating on this broader set provides a measure of out-of-distribution generalisation. Its window index is frozen; no post-hoc filtering or re-sampling is applied. The incremental training curriculum across the three stages is described in Chapter **??**.

3.2 Three-Branch Experimental Framework

The central question of this study is whether a latent world model improves candidate selection quality when paired with an imitation-learning policy, and whether that effect generalises across policy families. Answering both questions cleanly requires holding every other experimental factor constant. The benchmark therefore organises evaluation around three branches that share the same policy weights, the same evaluation windows, and the same candidate pool across B2 and B3; only the selection rule applied to the candidate pool differs (Figure 3.3). Any measured performance difference between branches is directly attributable to the selection mechanism rather than to training data, architecture, or window coverage.

Two policy families are evaluated within this framework: ACT, whose CVAE-Transformer decoder generates five candidate action chunks in a single forward pass, and Diffusion Policy (DP), which generates five candidates via five independent Denoising Diffusion Implicit Models (DDIM) denoising chains on the same observation. Both families are given equal standing; the candidate pool contract is identical for each, and all three branches apply the same selection rules to both.

Branch 1 (B1): Policy-only baseline. The policy generates its full candidate pool and selects the candidate it scores highest by its own internal preference. No world model is queried at runtime. B1 establishes the trajectory prediction quality the policy achieves acting on its own judgement alone and serves as the baseline against which both world-model branches are measured.

Branch 2 (B2): Hybrid with world-model veto. The policy generates the same five candidate action chunks. The world model scores all five by predicting forward in its latent space from the current observation and measuring the fidelity of each prediction. If the world model's preferred candidate differs from the policy's preferred candidate and its score exceeds the policy's choice by a declared margin threshold, the world model overrides the selection; otherwise the policy's preferred candidate stands. This conservative design confines world-model authority to windows where it has sufficient relative confidence. The mechanism is analogous in spirit to model-based offline reinforcement learning approaches such as MOReL [39] and MOPO [38], which

use a learned model's uncertainty to constrain policy decisions to regions where the model is reliable.

Branch 3 (B3): World-model-driven selection. The margin threshold is removed entirely. The world model selects the candidate with the highest score unconditionally. Critically, B3 does not re-run the policy: it operates over the frozen candidate pool produced during the B2 pass for each window. The policy contributes its proposals at B2 time; B3 reuses them without generating any new candidates. This design ensures that comparing B2 against B3 isolates the effect of removing the threshold gate on identical inputs, with no other variable changing.

The isolation property holds at every level of the evaluation. All 7,607 test windows are fixed and shared across all branches and all five world model families. The five-candidate pool per window is generated once and reused across B2 and B3. World model weights are fixed at the end of training and not updated during evaluation. The only degree of freedom is the selection rule applied to the pool. Differences in measured trajectory prediction quality between branches therefore reflect the causal effect of world model involvement in selection.

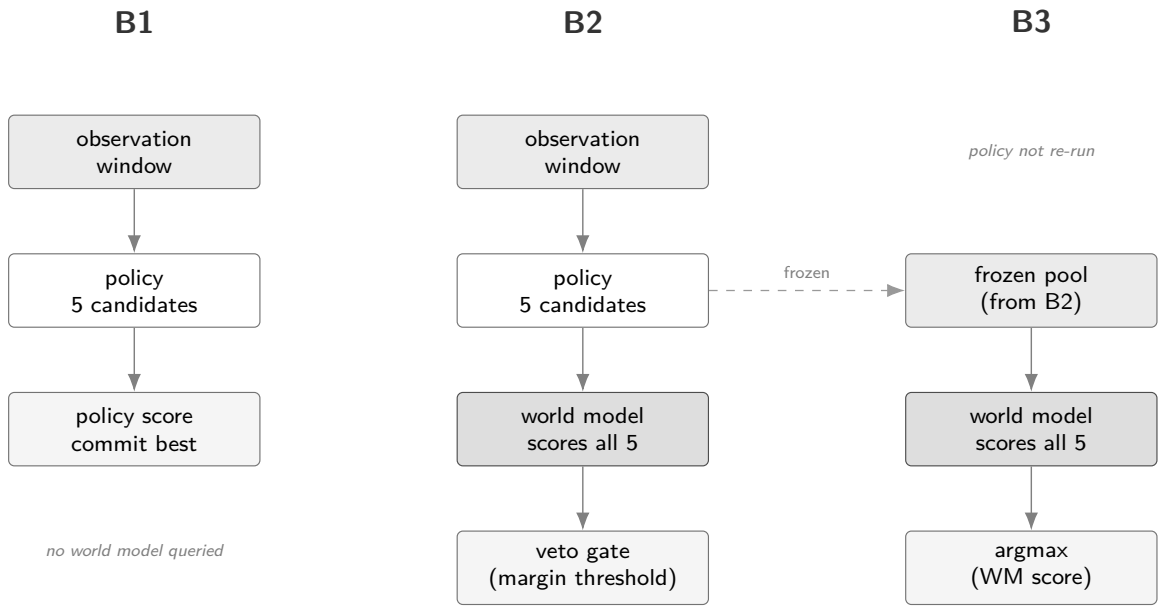


Figure 3.3 Three-branch design space. B1 uses the policy's own preference alone; B2 adds a world-model veto constrained by a margin threshold; B3 gives the world model full selection authority over the frozen candidate pool produced at B2 time. The policy is not re-run in B3.

3.3 Policy Variants

The benchmark evaluates two imitation-learning policy families that differ fundamentally in how they generate candidate action chunks. This difference is not

incidental: the two families occupy opposite ends of a spectrum in candidate generation mechanism, which makes any shared effect of world-model augmentation across both families a stronger and more general finding.

3.3.1 Action Chunking Transformer

The architecture and training objective of ACT are reviewed in Section 2.1. In the context of this benchmark, the relevant property is how ACT generates its candidate pool. The CVAE latent space is sampled five times and each draw is decoded in a single forward pass, yielding five candidate chunks simultaneously. The candidate pool therefore requires exactly one network evaluation regardless of pool size, making candidate generation computationally efficient. The input is a single RGB frame paired with the current bilateral wrist SE(3) state; the output is a 16-step bimanual wrist-transform sequence matching the prediction horizon of the windowing contract.

3.3.2 Diffusion Policy

The architecture and training objective of DP are reviewed in Section 2.2. The Diffusion Transformer (DiT) backbone variant is used here, as it has been shown to outperform convolutional UNet variants on manipulation benchmarks. The input and output contract is identical to that of ACT: a single RGB frame with bilateral wrist state as input, a 16-step SE(3) bimanual action chunk as output.

The candidate generation mechanism differs fundamentally from ACT. Each candidate requires one independent denoising chain; producing five candidates therefore demands five separate reverse-diffusion passes, each using an accelerated DDIM [45] schedule of ten steps. This yields approximately fifty network evaluations per window, roughly an order of magnitude more than the equivalent ACT forward pass. The candidates are structurally independent: each starts from a fresh noise sample, so diversity arises from stochastic initialisation rather than from a shared latent manifold.

3.3.3 Complementary coverage

The two families differ in latent structure, generation cost, and candidate diversity in ways that are relevant to how a world model might discriminate between proposals. ACT candidates are structured variations within a learned latent manifold; all five share the same observation-conditioned prior and their diversity is bounded by the width of that prior. DP candidates are independent draws from the full diffusion distribution; diversity is higher but the candidates carry less mutual structure.

Using both families under the same three-branch evaluation protocol allows the study to separate two questions: whether world-model selection improves over the baseline at all, and whether the magnitude of that improvement depends on how the candidate pool was generated. A result that holds across both families constitutes stronger evidence for the generality of the world-model-augmented selection mechanism.

3.4 World Model Families

3.4.1 V-JEPA2-AC — Pretrained Backbone Baseline

3.4.2 LeWM — Lightweight From-Scratch

3.4.3 DexWM — Domain-Specific Architecture

3.4.4 V-JEPA2 (No Action Conditioning) — Ablation Control

3.4.5 PointWorld — Out-of-Domain Negative Control

3.5 Fair Experimental Protocol

3.6 Evaluation Metrics and Score Harmonisation

3.7 Evaluation Scope and Limitations

References

[1] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009. DOI: 10.1016/j.robot.2008.10.024

[2] M. Jeannerod, "The timing of natural prehension movements," *Journal of Motor Behavior*, vol. 16, no. 3, pp. 235–254, 1984. DOI: 10.1080/00222895.1984.10735319

[3] M. Jeannerod, M. A. Arbib, G. Rizzolatti, and H. Sakata, "Grasping objects: The cortical mechanisms of visuomotor transformation," *Trends in Neurosciences*, vol. 18, no. 7, pp. 314–320, 1995. DOI: 10.1016/0166-2236(95)93921-J

[4] M. Santello, M. Flanders, and J. F. Soechting, "Postural hand synergies for tool use," *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998. DOI: 10.1523/JNEUROSCI.18-23-10105.1998

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, 2011.

[6] Z. Ding, "Imitation learning," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*, H. Dong, Z. Ding, and S. Zhang, Eds. Singapore: Springer Singapore, 2020, pp. 273–306, ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_8 [Online]. Available: https://doi.org/10.1007/978-981-15-4095-0_8

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, *Learning fine-grained bimanual manipulation with low-cost hardware*, Apr. 2023. DOI: 10.48550/arXiv.2304.13705 arXiv: 2304.13705 [cs].

[8] C. Chi et al., *Diffusion policy: Visuomotor policy learning via action diffusion*, Mar. 2024. DOI: 10.48550/arXiv.2303.04137 arXiv: 2303.04137 [cs].

[9] S. Jain et al., *DiWA: Diffusion policy adaptation with world models*, ArXiv preprint, Aug. 2025. arXiv: 2508.03645 [cs].

[10] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[11] Omitram, *Left-hand vs. right-hand coordinate systems: The 3d developer's guide - omitram — omitram.com*, https://omitram.com/3d-coordinate-systems-left-vs-right-hand/, 2024.

[12] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753. DOI: 10.1109/CVPR.2019.00589

[13] N. Grossmann, E. Gröller, and M. Waldner, "Concept splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2022.04.013 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849322000656

[14] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path towards autonomous machine intelligence," *Journal of Statistical Mechanics: Theory and Experiment*, 2023. arXiv: 2306.02572 [cs].

[15] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. DOI: 10.48550/arXiv.1312.6114 arXiv: 1312.6114 [stat.ML].

[16] L. Li and R. Betti, "A machine learning-based data augmentation strategy for structural damage classification in civil infrastructure system," *Journal of Civil Structural Health Monitoring*, vol. 13, pp. 1–21, May 2023. DOI: 10.1007/s13349-023-00705-5

[17] AnotherDatum. "Variational autoencoders explained," Accessed: Apr. 24, 2026. [Online]. Available: https://anotherdatum.com/vae.html

[18] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[19] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. DOI: 10.48550/arXiv.1706.03762 arXiv: 1706.03762 [cs.CL].

[20] S. Panda, *Decoding attention types and strategies for effective nlp models*, https://www.linkedin.com/pulse/decoding-attention-types-strategies-effective-nlp-models-swagat-panda-erhef/, 2024.

[21] M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv: 2301.08243 [cs].

[22] IBM Developer, *Models for machine learning*, https://developer.ibm.com/articles/cc-models-machine-learning/, 2017. Accessed: May 14, 2026.

[23] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, *Film: Visual reasoning with a general conditioning layer*, 2017. arXiv: 1709.07871 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1709.07871

[24] *MTDP: Modulated Transformer Diffusion Policy Model*, https://arxiv.org/html/2502.09029v1. Accessed: Apr. 26, 2026.

[25] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, *Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning*, 2024. arXiv: 2412.12953 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2412.12953

[26] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, *Mastering diverse domains through world models*, Nature, 2025, 2023. arXiv: 2301.04104 [cs].

[27] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang, *WorldGym: World model as an environment for policy evaluation*, Sep. 2025. DOI: 10.48550/arXiv.2506.00613 arXiv: 2506.00613 [cs].

[28] Q. Bu et al., *Closed-loop visuomotor control with generative expectation for robotic manipulation*, Oct. 2024. DOI: 10.48550/arXiv.2409.09016 arXiv: 2409.09016 [cs].

[29] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022. arXiv: 2203.12601 [cs].

[30] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2212.06817

[31] M. Assran et al., *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning*, Jun. 2025. DOI: 10.48550/arXiv.2506.09985 arXiv: 2506.09985 [cs].

[32] N. Tsagkas, A. Sochopoulos, D. Danier, C. X. Lu, and O. Mac Aodha, *When pre-trained visual representations fall short: Limitations in visuo-motor robot learning*, 2025. arXiv: 2502.03270 [cs].

[33] R. G. Goswami et al., *World models can leverage human videos for dexterous manipulation*, Dec. 2025. DOI: 10.48550/arXiv.2512.13644 arXiv: 2512.13644 [cs].

[34] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, *Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels*, Verified from local PDF metadata and first page, Mar. 2026. arXiv: 2603.19312 [cs.LG].

[35] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, *DINO-WM: World models on pre-trained visual features enable zero-shot planning*, 2024. arXiv: 2411.04983 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.04983

[36] W. Huang et al., *Pointworld: Scaling 3d world models for in-the-wild robotic manipulation*, 2026. arXiv: 2601.03782 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2601.03782

[37] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.16828 [cs].

[38] T. Yu et al., "MOPO: Model-based offline policy optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: 2005.13239 [cs].

[39] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-based offline reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: 2005.05951 [cs].

[40] A. Argenson and G. Dulac-Arnold, "Model-based offline planning," in *International Conference on Learning Representations (ICLR)*, 2021. arXiv: 2008.05556 [cs].

[41] Z. Liu, T. Swann, S. Garg, S. Levine, and A. Kumar, *Model-based offline planning with trajectory pruning*, 2021. arXiv: 2105.07351 [cs].

[42] X. Li et al., *Evaluating real-world robot manipulation policies in simulation*, CoRL 2024, 2024. arXiv: 2405.05941 [cs].

[43] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, *WorldEval: World model as real-world robot policies evaluator*, 2025. arXiv: 2505.19017 [cs].

[44] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang, *EgoDex: Learning dexterous manipulation from large-scale egocentric video*, ICLR 2026, 2025. DOI: 10.48550/arXiv.2505.11709 arXiv: 2505.11709 [cs].

[45] J. Song, C. Meng, and S. Ermon, *Denoising diffusion implicit models*, Oct. 2020. DOI: 10.48550/arXiv.2010.02502 arXiv: 2010.02502 [cs.LG].